

Retrospective Sprint 4 of Group UG13

Block Model Compression Algorithm

Team Members:

1. a1837374 Mengbo Ren
2. a1837306 Suchang Liu
3. a1824243 Yuyao Chen
4. a1837254 Yang Shi
5. a1832623 Hin Kai Chan
6. a1860970 Mohsan Salehi-Pugsley
7. a1818124 Chih-Hao Chang

Snapshots

I attended the sprint review/planning meeting on the 16/Oct with the tutor.

Snapshots are provided in the appendix at the end of the retrospective.

What went well in the sprint?

In the last sprint, our team focused on the final deliverables of the project, including optimization, debugging, and final documentation.

The first thing that went well in the last run was better time management of task implementations and submission. Almost all major tasks in the development phase have been refined and completed. The team's work has been validated on the Titan testing Machine. This is obvious, but it was really important for the overall project delivery. In the first phase, with all the basic work done, the team will had time for more extensive testing and internal optimization. For example, the team was able to perform black box testing as regular users to find user experience areas to better enhance the code base. Hence, the team was able to ensure the highest quality of the software along with getting more feedback on the new beta via Slack communication by tracking the final project release rate. This was very crucial because the customer could fully test the demo version of the final product and quickly recommend the necessary changes such as minor improvements to the user interface which then ensured that the team has enough time to implement customer recommendations that meet customer expectations for the final product, resulting in greater customer satisfaction.

In this fourth run, the project phase changed from the development phase to the delivery phase (including optimization). Here, I believe when the team's focus changes, a quick team planning meeting is important to discuss goals and actions for the final run. This meeting was also a highlight from the last sprint. In this phase, the team planned on developing new tasks such as adding more resources to the research report for more information from the research team, the testing team creating more black-box tests, the code factorization, and the development team benchmarking which are to finalize the functional and non-functional needs of the proxy client.

What could be improved?

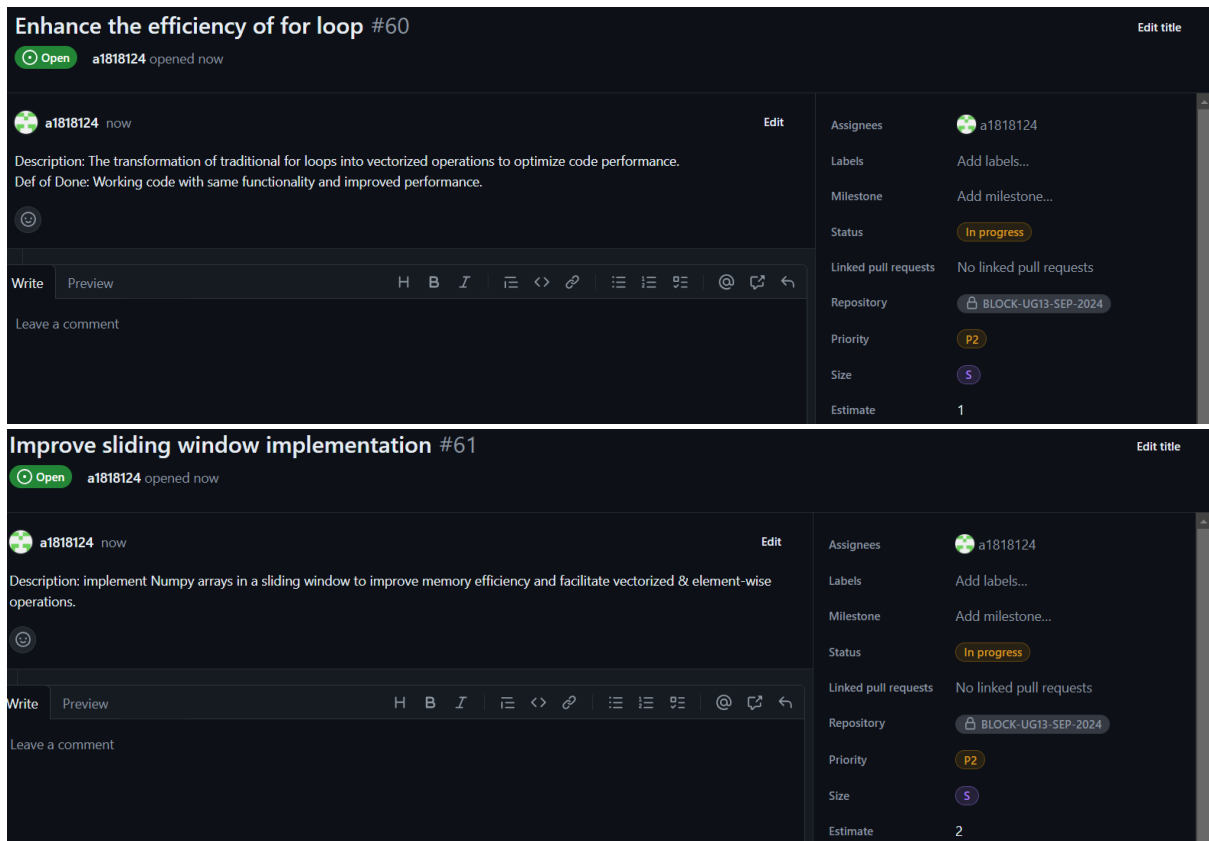
In addition to the effective planning for the final delivery of the software, there are several areas that needed enhancement. Initially, the team failed to foresee that the testing machine would be unavailable significantly earlier than the project's conclusion. Consequently, the team had to allocate some time from the optimization phase to finalize Octree alternatives aimed at improving their leaderboard score. This decision was deemed a reasonable tradeoff, as the team achieved a commendable ranking. Nonetheless, it would have been prudent for the team to factor in the closure date of the testing machine to enhance time management.

The team has completed the research reports on GitHub Wiki; however, they overlooked the documentation concerning the comparison of software speedup techniques, including dynamic programming, multi-threading, and GPU utilization. Additionally, due to the premature closure of the Titan testing machine, the team was unable to gather comprehensive records of their submissions, which would have facilitated a more accurate visualization of speedup. These reports could serve as valuable references for the client during future system scaling efforts, thereby addressing some of the client's non-functional requirements related to scalability. Fortunately, this issue can be rectified swiftly through a simple script and the current test cases, which will be elaborated upon in the subsequent discussion.

Lastly, the allocation of time for various tasks could be improved. The team's concentration on refining the compression algorithm left limited time for exploring alternative experiments aimed at enhancing software speedup. For instance, there is currently only one implementation utilizing CUDA GPU. Therefore, if additional resources had been dedicated to software speedup, the team could have developed a broader array of alternatives, thereby accommodating a wider user base and offering enhanced performance options.

What will the group commit to improve in the next sprint?

To enhance software scalability, our team is currently prioritizing a report on performance improvements from multi-threading, dynamic programming, and GPU utilization. While data from the Titan testing machine will be used as a reference, it is somewhere outdated, requiring new manual experiments. The testing team will utilize existing test cases and scripts to measure CPU time for each solution, while also automating the testing process. The research team will analyze the results for visual representation. This report will address the client's non-functional requirements and guide them in prioritizing speedup methods for improvement.



In the final sprint of the project, my primary objective was to finalize the implementation of the Octree data structure, which is essential for enabling efficient tracebacks and facilitating multi-threading compression. This undertaking was one of the most challenging and time-intensive tasks persisted for me since the development phase. Additionally, I am in the process of completing the research reports on the GitHub Wiki and enhancing performance through the refactoring of the existing codebase.

The implementation of the Octree necessitates the establishment of Octree data structures along with fundamental operations such as insertion, deletion, and retrieval of specific patterns based on input types. The development team has divided this task into the implementation of various sub-routines within the Octree framework. My specific focus within the Octree project encompasses the creation of the Octree (insertion and deletion) and the traceback for compression (retrieval).

I successfully executed the Octree creation at the outset of the run-length encoding process, subsequently adding new nodes or blocks into the Octree until the completion of the third dimension slice. Once all requisite slices have been processed by the run-length encoding for block storage in the Octree, the retrieval operation can be invoked to identify suitable blocks for constructing a cohesive block across multiple third dimension slices.

The merging of two blocks from different slices (two distinct Octrees) has been accomplished by my colleagues. Consequently, the final step in achieving high-performance third-dimension compression involves assessing which blocks should

be merged to optimize the compression ratio. To facilitate this, I developed an additional code segment to dynamically evaluate compression performance. For each candidate block formation, the code prioritizes those with the highest number of input blocks. However, if the compression performance does not show improvement, the algorithm will explore alternative blocks. It is also important to note that attempting to improve the compression ratio by evaluating every block is not a prudent strategy. Therefore, I restricted the algorithm to consider only the five largest blocks when constructing blocks across slices. This limitation allowed the algorithm to enhance the compression ratio through evaluation while maintaining speed, as the number of attempts is kept minimal.

In addition, I also prioritized the refactoring of the codebase prior to the final submission. For the Python codebase, I undertook a refactoring process aimed at enhancing performance. Specifically, I addressed the inefficiencies associated with "for loops" in Python, which stem from the language's lack of concrete types, necessitating decision-making at each instruction. To resolve this, I replaced for loops with vectorized operations. These operations facilitated element-wise computations across entire N-dimensional arrays with a single function call, eliminating the need for iterative processing of each element. Furthermore, I transitioned from using lists for the manual sliding window to employing Numpy arrays. This shift not only leverages the benefits of vectorization, as Numpy arrays support numerous vectorized operations, but also enhances memory efficiency compared to traditional Python lists.

Appendix

Snapshot Week 9 of Group **Block-UG13**

Block Model Compression Algorithm

Team members:

a1837374 Mengbo Ren

a1837306 Suchang Liu

a1824243 Yuyao Chen

a1837254 Yang SHI

a1832623 Hin Kai Chan

a1860970 Mohsan Salehi-Pugsley

a1818124 Chih-Hao Chang

a1881772 Nauman Yawar Butt

Product Backlog and Task Board:

The product backlog

Product backlog

2 / 3

...

Estimate: 7

This item hasn't been started

BLOCK-UG13-SEP-2024 #31

Develop an algorithm for a $n*n*n$

P0

L

3

↻ sprint 4

BLOCK-UG13-SEP-2024 #30

Bug Fixes

M

4

↻ sprint 4

+ Add item

The task board

To do

2

Estimate: 13

...

This is ready to be picked up

BLOCK-UG13-SEP-2024 #36

Using 2*2*2 blocks to compress

P0

M

5

sprint 4

BLOCK-UG13-SEP-2024 #40

Using 4*4*1 blocks to compress

P1

M

8

sprint 4

+ Add item

 **In progress** 3 / 3 Estimate: 13 

This is actively being worked on

 BLOCK-UG13-SEP-2024 #18 

Documentation Retrospective Sprint 4

 P2  L  3  sprint 4

 BLOCK-UG13-SEP-2024 #20 

input stream processor update

 P1  M  5  sprint 4

 BLOCK-UG13-SEP-2024 #21 

Error handling

 P1  M  5  sprint 4

+ Add item

 **In review** 3 / 5 Estimate: 12 

This item is in review

 BLOCK-UG13-SEP-2024 #33 

Maptek competition

P0

L

5

 sprint 4

 BLOCK-UG13-SEP-2024 #39 

Debug running shell, script

P0

M

5

 sprint 4

 Draft 

Snapshot Week 9

P2

L

2

 sprint 4

+ Add item

Sprint 4 Done

3

Estimate: 15

...

This has been completed in sprint 4

BLOCK-UG13-SEP-2024 #24

Methods and configurations for compiling an exe executed in Windows in vscode cmd

P0

M

4

🔄 sprint 4

BLOCK-UG13-SEP-2024 #32

algorithm for a 2*2*1 model output

P0

S

6

🔄 sprint 4

BLOCK-UG13-SEP-2024 #37

Understanding 2024_runnerA.py Testing

P0

M

5

🔄 sprint 3

+ Add item

Sprint Backlog and User Stories:

Sprint backlog

Sprint Backlog

3

Estimate: 10

...

BLOCK-UG13-SEP-2024 #27

New features

P2

S

3

↻ sprint 4

BLOCK-UG13-SEP-2024 #28

Enhancement

P2

L

4

↻ sprint 3

BLOCK-UG13-SEP-2024 #29

Technical Debt

P2

M

3

↻ sprint 3

+ Add item

user stories

Sprint Overview

For the current sprint, we've chosen the following user stories to focus on:

1.As a product manager, we hope that the product can be launched as soon as possible, so we take the realization of the algorithm as the first goal in this sprint.

2.As developers, the testing process was very repetitive and tedious, so we wrote a debug script.

These user stories and their associated tasks will guide our current sprint development efforts, ensuring that we meet the needs of developers and customers as we create an effective block model compression program.

Definition of Done

1 Code complete:

Complete the code for the above three user use cases

2 Tests passed:

The code passed the local test, conforming to logic and the output met

expectations.

Summary of Changes:

1. The code uses standard input (stdin) to read data instead of specifying a file name.
2. Modified the algorithm implementation method

Snapshot Week 10 of Group **Block-UG13**

Block Model Compression Algorithm

Team members:

a1837374 Mengbo Ren

a1837306 Suchang Liu

a1824243 Yuyao Chen

a1837254 Yang SHI

a1832623 Hin Kai Chan

a1860970 Mohsan Salehi-Pugsley

a1818124 Chih-Hao Chang

a1881772 Nauman Yawar Butt

Product Backlog and Task Board:

The product backlog

Product backlog

3

...

This item hasn't been started

BLOCK-UG13-SEP-2024 #31

Develop an algorithm for a $n*n*n$

L

BLOCK-UG13-SEP-2024 #30

Bug Fixes

M

BLOCK-UG13-SEP-2024 #46

High speed input stream

L

+ Add item

The task board

To do

2

...

This is ready to be picked up

BLOCK-UG13-SEP-2024 #44

26-tree structure for compression

L

BLOCK-UG13-SEP-2024 #45

build_and_run.cmd

XS

+ Add item

 In progress 3



This is actively being worked on

 BLOCK-UG13-SEP-2024 #21



Error handling



 BLOCK-UG13-SEP-2024 #36



Using 2*2*2 blocks to compress



 BLOCK-UG13-SEP-2024 #40



Using 4*4*1 blocks to compress



+ Add item

 In review 2



This item is in review

 BLOCK-UG13-SEP-2024 #18



Documentation Retrospective Sprint 4



 BLOCK-UG13-SEP-2024 #20



input stream processor update



+ Add item

Sprint 4 Done

5

This has been completed in sprint 4

BLOCK-UG13-SEP-2024 #39

Debug running shell, script

M

BLOCK-UG13-SEP-2024 #33

Maptek competition

L

BLOCK-UG13-SEP-2024 #32

algorithm for a 2*2*1 model output

S

BLOCK-UG13-SEP-2024 #24

Methods and configurations for compiling an exe executed in Windows in vscode cmd

M

BLOCK-UG13-SEP-2024 #37

Understanding 2024_runnerA.py Testing

M

Add item

Sprint Backlog and User Stories:

Sprint backlog

<

user stories

Sprint Overview

For the current sprint, we've chosen the following user stories to focus on:

1. Since the last sprint, the focus of work of the team has been reassigned, and my work is mainly to write code. I wrote several versions and uploaded the website test, in which Block_UG13_1004.exe passed and was listed in the ranking.

2.As developers, the testing process was very repetitive and tedious, so we wrote a debug script:Build_and_run.cmd . After two weeks of testing, modification, and streamlining, it has been uploaded to Github for the group's partners to use.

These user stories and their associated tasks will guide our current sprint development efforts, ensuring that we meet the needs of developers and customers as we create an effective block model compression program.

Definition of Done

1 Code complete:

Complete the code for the above three user use cases

2 Tests passed:

The code passed the local test, conforming to logic and the output met expectations.

Summary of Changes:

1. Modified the version, the performance is not up but down, no upload.
2. Debug running shell, script.

We first install VScode in Windows11, by opening virtualization and installing WSL and Ubuntu, install g++ compiler in Linux, exe compiled under Linux can not run in the website, but debugging and running more familiar and easy. So we first compile and run it under Linux to make the code bug-free. Because of the heavy debugging, we write the debugging process as a shell script. Perform batch data testing, compare compression rate and speed, and find performance improvement points.

3. To perform 3*3*3 compression, a data structure is required to easily obtain the state of 26 directional neighbor points.